

Assignment #B: 图 (1/4)

Updated 2031 GMT+8 Nov 17, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E07218: 献给阿尔吉侬的花束

bfs, <http://cs101.openjudge.cn/practice/07218/>

思路：

BFS搜索：从起点S开始，逐层向外扩展，第一次到达终点E时的步数就是最短路径

四个方向：每次可以向上下左右四个方向移动

访问标记：使用visited数组标记已访问的位置，避免重复访问

队列存储：队列中存储当前位置和已走步数

代码：

```

from collections import deque

def solve_maze(R, C, maze):
    start = None
    end = None
    for i in range(R):
        for j in range(C):
            if maze[i][j] == 'S':
                start = (i, j)
            elif maze[i][j] == 'E':
                end = (i, j)

    queue = deque([(start[0], start[1], 0)])
    visited = [[False] * C for _ in range(R)]
    visited[start[0]][start[1]] = True
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        x, y, steps = queue.popleft()
        if x == end[0] and y == end[1]:
            return steps
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if (0 <= nx < R and 0 <= ny < C and
                not visited[nx][ny] and maze[nx][ny] != '#'):
                visited[nx][ny] = True
                queue.append((nx, ny, steps + 1))
    return -1

T = int(input())
for _ in range(T):
    R, C = map(int, input().split())
    maze = []
    for i in range(R):
        maze.append(input().strip())

    result = solve_maze(R, C, maze)
    if result == -1:
        print("oop!")
    else:
        print(result)

```

代码运行截图（至少包含有"Accepted"）

#50981297提交状态

状态: Accepted

源代码

M27925: 小组队列

dict, queue, <http://cs101.openjudge.cn/practice/27925/>

思路:

数据结构设计:

使用字典记录每个人所属的小组

使用队列记录小组的顺序

为每个小组维护一个队列，记录该小组成员的入队顺序

ENQUEUE操作:

如果该人所属小组已在队列中，将其加入该小组的队列末尾

否则，创建新的小组队列并加入整体队列

DEQUEUE操作:

从队首小组中取出第一个人

如果该小组为空，将其从队列中移除

代码:

```

from collections import deque, defaultdict
t = int(input())
person_to_group = {}
for group_id in range(t):
    members = list(map(int, input().split()))
    for person in members:
        person_to_group[person] = group_id
group_queues = defaultdict(deque)
main_queue = deque()
group_in_queue = set()
while True:
    command = input().split()

    if command[0] == "STOP":
        break

    elif command[0] == "ENQUEUE":
        person = int(command[1])
        if person in person_to_group:
            group = person_to_group[person]
        else:
            group = ("散客", person)
        if group not in group_in_queue:
            main_queue.append(group)
            group_in_queue.add(group)
            group_queues[group].append(person)

    elif command[0] == "DEQUEUE":
        if main_queue:
            first_group = main_queue[0]
            person = group_queues[first_group].popleft()
            print(person)
            if not group_queues[first_group]:
                main_queue.popleft()
                group_in_queue.remove(first_group)

```

代码运行截图（至少包含有"Accepted"）

#50981401提交状态

状态: Accepted

提交时间

M04089: 电话号码

trie, <http://cs101.openjudge.cn/practice/04089/>

思路:

对所有号码排序

只需检查相邻的号码是否有前缀关系（因为如果A是C的前缀，排序后A和C之间的B也会是A的前缀或C的前缀）

代码:

```
def check_consistency(phone_numbers):
    phone_numbers.sort()
    for i in range(len(phone_numbers) - 1):
        if phone_numbers[i + 1].startswith(phone_numbers[i]):
            return False

    return True

t = int(input())
for _ in range(t):
    n = int(input())
    phone_numbers = []
    for _ in range(n):
        phone = input().strip()
        phone_numbers.append(phone)

    if check_consistency(phone_numbers):
        print("YES")
    else:
        print("NO")
```

代码运行截图（至少包含有"Accepted"）

#50981494提交状态

状态: Accepted

M3532.针对图的路径存在性查询I

disjoint set, <https://leetcode.cn/problems/path-existence-queries-in-a-graph-i/>

思路:

由于 nums 有序, 如果节点 i 和 j 能通过一系列中间节点连通, 我们不需要直接连接它们。只需要连接"相邻"的满足条件的节点即可。

对每个节点, 只连接它后面第一个满足条件的节点, 而不是所有满足条件的节点。通过传递性, 所有应该连通的节点最终都会连通。

代码

```
class Solution(object):
    def pathExistenceQueries(self, n, nums, maxDiff, queries):
        parent = list(range(n))

        def find(x):
            if parent[x] != x:
                parent[x] = find(parent[x])
            return parent[x]

        def union(x, y):
            root_x = find(x)
            root_y = find(y)
            if root_x != root_y:
                parent[root_x] = root_y

        for i in range(n - 1):
            if nums[i + 1] - nums[i] <= maxDiff:
                union(i, i + 1)

        result = []
        for u, v in queries:
            result.append(find(u) == find(v))

        return result
```

代码运行截图（至少包含有"Accepted"）



M19943: 图的拉普拉斯矩阵

OOP, graph, implementation, <http://cs101.openjudge.cn/pctbook/E19943/>

要求创建Graph, Vertex两个类，建图实现。

思路：

Vertex 类：

id：顶点编号

neighbors：存储相邻顶点的集合

add_neighbor()：添加相邻顶点

get_degree()：返回度数（相邻顶点数量）

is_connected()：判断是否与指定顶点相连

Graph 类：

n：顶点数量

vertices：存储所有顶点的字典

add_edge()：添加无向边（双向连接）

get_degree_matrix()：生成度数矩阵 D

get_adjacency_matrix()：生成邻接矩阵 A

get_laplacian_matrix()：计算拉普拉斯矩阵 $L = D - A$

拉普拉斯矩阵特点：

对角线元素：顶点的度数（正值）

非对角线元素：如果有边则为 -1，否则为 0

每行和为 0

代码

```

class Vertex:
    def __init__(self, id):
        self.id = id
        self.neighbors = set()

    def add_neighbor(self, neighbor_id):
        self.neighbors.add(neighbor_id)

    def get_degree(self):
        return len(self.neighbors)

    def is_connected(self, other_id):
        return other_id in self.neighbors


class Graph:
    def __init__(self, n):
        self.n = n
        self.vertices = {}
        for i in range(n):
            self.vertices[i] = Vertex(i)

    def add_edge(self, a, b):
        self.vertices[a].add_neighbor(b)
        self.vertices[b].add_neighbor(a)

    def get_degree_matrix(self):
        D = [[0] * self.n for _ in range(self.n)]
        for i in range(self.n):
            D[i][i] = self.vertices[i].get_degree()
        return D

    def get_adjacency_matrix(self):
        A = [[0] * self.n for _ in range(self.n)]
        for i in range(self.n):
            for j in range(self.n):
                if i != j and self.vertices[i].is_connected(j):
                    A[i][j] = 1
        return A

    def get_laplacian_matrix(self):
        D = self.get_degree_matrix()
        A = self.get_adjacency_matrix()

```



```

# L = D - A
L = [[0] * self.n for _ in range(self.n)]
for i in range(self.n):
    for j in range(self.n):
        L[i][j] = D[i][j] - A[i][j]

return L

```

```
n, m = map(int, input().split())
```

```
graph = Graph(n)
```

```

for _ in range(m):
    a, b = map(int, input().split())
    graph.add_edge(a, b)

```

```
laplacian = graph.get_laplacian_matrix()
```

```

for row in laplacian:
    print(' '.join(map(str, row)))

```

代码运行截图（至少包含有"Accepted"）

#50981731提交状态

状态: Accepted

T25353: 排队

<http://cs101.openjudge.cn/pctbook/T25353/>

思路:

代码:

代码运行截图（至少包含有"Accepted"）

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

图是一种能高效表达复杂关系的核心结构；通过掌握不同表示方式（邻接矩阵/邻接表）与关键算法（遍历、最短路、拓扑排序、最小生成树等），能够将现实中的关系、网络与依赖问题抽象成可计算、可解决的形式，从而显著提升建模与算法设计能力。